

# Homework5

February 25, 2019

```
In [1]: import numpy as np
import sklearn as sk
from sklearn import model_selection
import scipy.stats
import pandas as pd
import matplotlib
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.calibration import calibration_curve
from sklearn.metrics import confusion_matrix
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis
from sklearn.metrics import roc_curve
from sklearn.metrics import roc_auc_score
import pygam
from pygam import LinearGAM
from pygam import s, f
from pygam import LinearGAM
from pygam import LogisticGAM
from sklearn.ensemble import RandomForestClassifier
```

```
In [2]: import warnings
warnings.filterwarnings("ignore")
```

```
In [3]: matplotlib.rcParams['figure.figsize'] = (9,6)
```

## 1 Problem 1

```
In [4]: marketing = pd.read_csv('./marketing.csv')
market_data = marketing.transform(lambda x: LabelEncoder().fit_transform(x) if x.dtype
market_data['y'] = (marketing['y']=='yes').astype('int')
market_data.head()
market_data.head()
```

# Problem 1

February 24, 2019

```
x_train = read.table('xtrain.csv', sep=',', header=TRUE, row.names = 'X')
x_test = read.table('xtest.csv', sep=',', header=TRUE, row.names = 'X')
y_train = read.table('ytrain.csv', sep=',', row.names = 'V1')
y_test = read.table('ytest.csv', sep=',', row.names = 'V1')

train = cbind(as.factor(y_train$V2), x_train)
testdata = cbind(as.factor(y_test$V2), x_test)
names(train)[1] = 'y'
names(train)[1] = 'y'
colnames(train)

## [1] "y"          "age"        "job"        "marital"    "education" "default"
## [7] "balance"   "housing"    "loan"
```

(a)

```
library(rpart)
fit1 = rpart(y~. , method = "class", data = train)
summary(fit1)

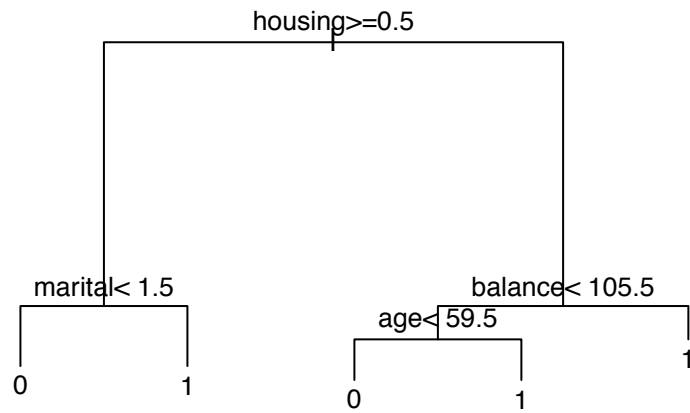
## Call:
## rpart(formula = y ~ ., data = train, method = "class")
##   n= 30291
##
##   CP nsplit rel error xerror xstd
## 1  0      0          1      0      0
##
## Node number 1: 30291 observations
##   predicted class=0 expected loss=0.1178238 P(node) =1
##   class counts: 26722 3569
##   probabilities: 0.882 0.118

#plot(fit)
```

Error message shows that the fit is not a tree, it is just a root, thus, we cannot plot it. By default, the rpart function won't make a split. Due to the heavily imbalanced data set, the number of default is very small compared to the whole data set.

(b)

```
wgts = rep(1, length(y_train$V2))
wgts[which(y_train == 1)] = 10
fit2 = rpart(y~. , method = "class", data = train, weights = wgts)
plot(fit2, margin = 0.2)
text(fit2, pretty=0, cex=0.8)
```



```
ypred_tree = predict(fit2, x_test, type = 'class')
print("Tree misclassification rate: ", mean(ypred_tree != y_test$V2) )
```

```
## [1] "Tree misclassification rate: "
```

```
mean(ypred_tree != y_test$V2)
```

```
## [1] 0.4418901
```

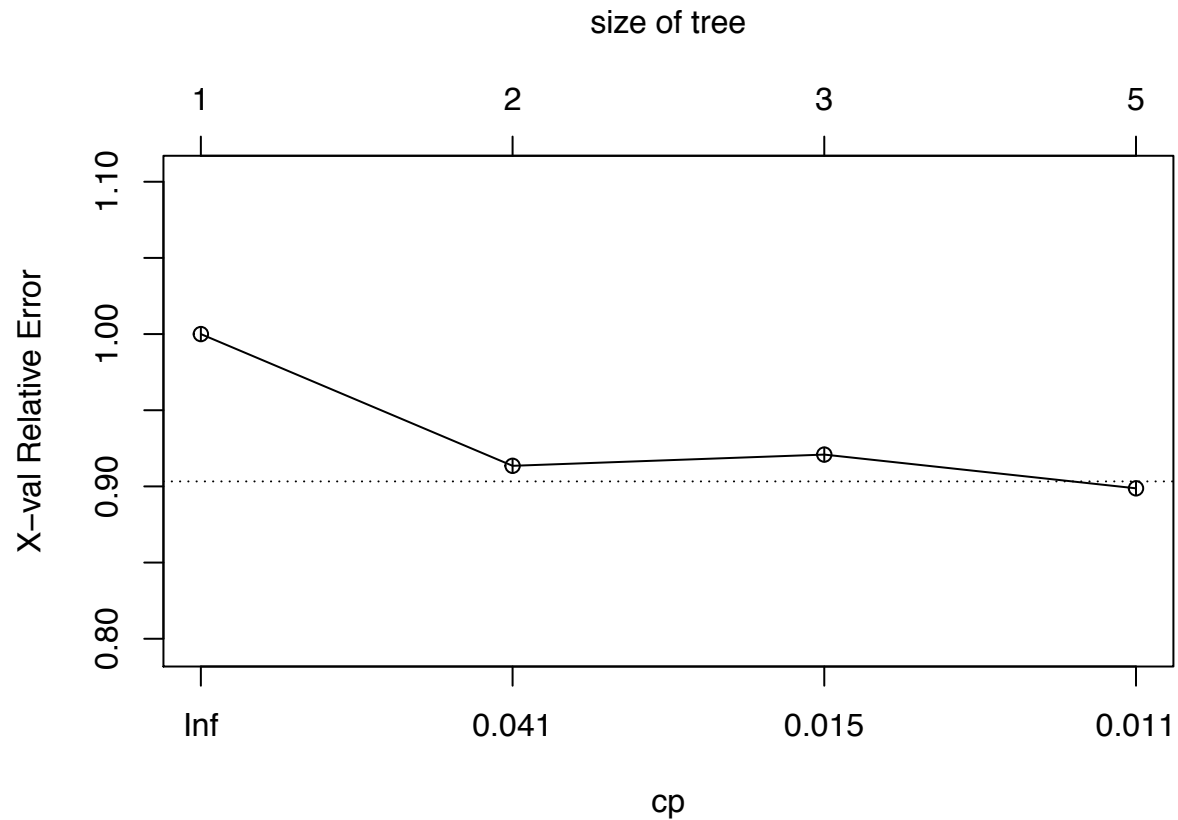
The misclassification rate is much larger than logistic regression (0.1154) and base rate(0.1178).

```
predict_prob_tree = predict(fit2, x_test, type = 'prob')[,2]
n = length(predict_prob_tree)
top_1000 = rownames(x_test)[order(predict_prob_tree)][(n-999):n]
sum(y_test[top_1000,])/1000
```

```
## [1] 0.178
```

(c)

```
plotcp(fit2)
```

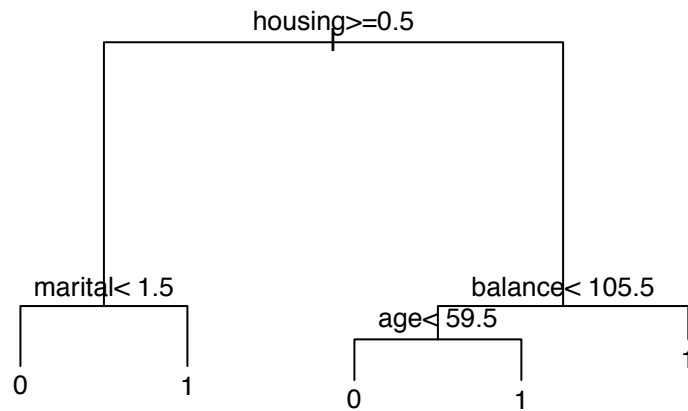


```
printcp(fit2)
```

```
##
## Classification tree:
## rpart(formula = y ~ ., data = train, weights = wgts, method = "class")
##
## Variables actually used in tree construction:
## [1] age      balance housing marital
##
## Root node error: 26722/30291 = 0.88218
##
## n= 30291
##
##      CP nsplit rel error  xerror    xstd
## 1 0.086483      0  1.00000 1.00000 0.0046260
## 2 0.019684      1  0.91352 0.91352 0.0045623
## 3 0.011433      2  0.89383 0.92089 0.0045688
## 4 0.010000      4  0.87097 0.89874 0.0045487
opt_cp = fit2$cptable[which.min(fit2$cptable[, "xerror"]), "CP"]
opt_cp
```

```
## [1] 0.01
```

```
prunetree = prune(fit2, cp=opt_cp)
plot(prunetree, margin = 0.2)
text(prunetree, pretty=0, cex=0.8)
```



```
predict_prob_prune = predict(prunetree, x_test, type = 'prob')[,2]
library(pROC)
```

```
## Type 'citation("pROC")' for a citation.
```

```
##
```

```
## Attaching package: 'pROC'
```

```
## The following objects are masked from 'package:stats':
```

```
##
```

```
## cov, smooth, var
```

```
logi_score = read.table('testprob.csv', sep=',', row.names = 'V1')
roc(y_test$V2, predict_prob_tree, plot=TRUE)
```

```
##
```

```
## Call:
```

```
## roc.default(response = y_test$V2, predictor = predict_prob_tree, plot = TRUE)
```

```
##
```

```
## Data: predict_prob_tree in 13200 controls (y_test$V2 0) < 1720 cases (y_test$V2 1).
```

```
## Area under the curve: 0.6359
```

```
roc(y_test$V2, predict_prob_prune, plot=TRUE, col='yellow', add=TRUE)
```

```
##
```

```
## Call:
```

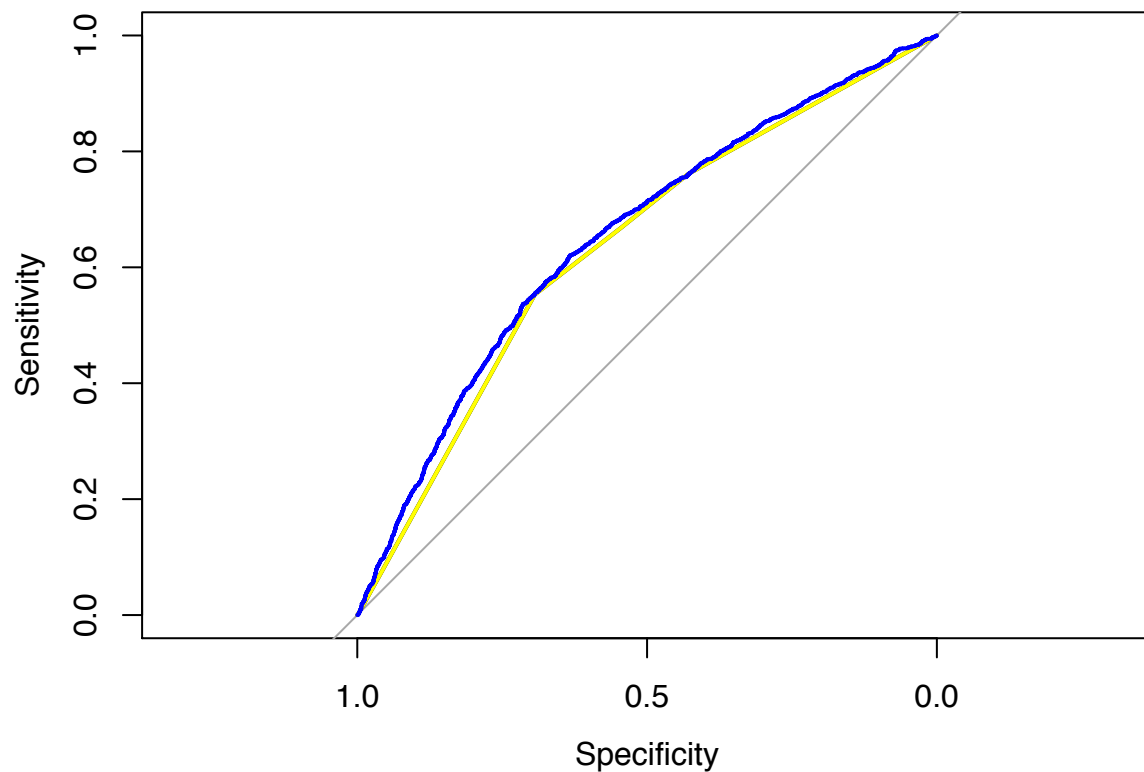
```
## roc.default(response = y_test$V2, predictor = predict_prob_prune, plot = TRUE, col = "yellow", a
```

```
##
```

```
## Data: predict_prob_prune in 13200 controls (y_test$V2 0) < 1720 cases (y_test$V2 1).
```

```
## Area under the curve: 0.6359
```

```
roc(y_test$V2, logi_score$V2, plot=TRUE, col='blue', add=TRUE)
```



```
##
## Call:
## roc.default(response = y_test$V2, predictor = logi_score$V2,      plot = TRUE, col = "blue", add = TRUE)
##
## Data: logi_score$V2 in 13200 controls (y_test$V2 0) < 1720 cases (y_test$V2 1).
## Area under the curve: 0.652
```

The decision process of trees are discrete, so the predicted scores can only be distributed on some certain sections. Thus, when we change the threshold, we are making linear changes in the classification results. So, the ROC curve for the two trees are piecewise linear.

```
Out [4]:
```

|   | age | job | marital | education | default | balance | housing | loan | y |
|---|-----|-----|---------|-----------|---------|---------|---------|------|---|
| 0 | 58  | 4   | 1       | 2         | 0       | 2143    | 1       | 0    | 0 |
| 1 | 44  | 9   | 2       | 1         | 0       | 29      | 1       | 0    | 0 |
| 2 | 33  | 2   | 1       | 1         | 0       | 2       | 1       | 1    | 0 |
| 3 | 47  | 1   | 1       | 3         | 0       | 1506    | 1       | 0    | 0 |
| 4 | 33  | 11  | 2       | 3         | 0       | 1       | 0       | 0    | 0 |

```
In [5]: X = market_data.drop(columns=['y'])
        Y = market_data['y']
        xtrain, xtest, ytrain, ytest = train_test_split(X, Y, test_size=0.33, random_state=1)
        xtrain.to_csv("/Users/rachel/Desktop/xtrain.csv")
        xtest.to_csv("/Users/rachel/Desktop/xtest.csv")
        ytrain.to_csv("/Users/rachel/Desktop/ytrain.csv")
        ytest.to_csv("/Users/rachel/Desktop/ytest.csv")
```

```
In [6]: log_reg = LogisticRegression(C=1000000).fit(xtrain, ytrain)
        ypred = log_reg.predict(xtest)
        logreg_misclass_rate = np.mean(ytest != ypred)
        base_rate = sum(ytrain)/ len(ytrain)
        print("Logistic Regression misclassification rate: ", logreg_misclass_rate)
        print("base rate: ", base_rate)
```

```
Logistic Regression misclassification rate: 0.11541554959785523
base rate: 0.11782377603908752
```

```
In [7]: test_prob = log_reg.predict_proba(xtest)[:,-1]
        pd.Series(test_prob).to_csv("/Users/rachel/Desktop/testprob.csv")
```

## 2 Problem 2

(a)(i)

```
In [8]: logam = LogisticGAM().gridsearch(xtrain.values, ytrain, lam = np.logspace(-5, 3, 30), 1
```

```
100% (30 of 30) |#####| Elapsed Time: 0:01:13 Time: 0:01:13
```

```
In [9]: log_y_pred = logam.predict(xtest)
        logypred_dens = logam.predict_proba(xtest)
        logam_mc_rate = np.mean(log_y_pred != ytest)
        top_1000 = np.argsort(test_prob)[:,-1][:1000]
        logam_yes_in_top_1000 = np.mean(np.array(ytest)[top_1000])
        logam_yes_in_top_1000
        print("Logistic GAM misclassification rate: ", logam_mc_rate)
```

```
#logistic regression
```

```
top_1000_logistic = np.argsort(test_prob)[:,-1][:1000]
```

```

log_success_in_top_1000 = np.mean(np.array(ytest)[top_1000_logistic])
print("Logistic success in top 1000: ", log_success_in_top_1000)
# logistic GAM top 1000
top_1000_logistic_logam = np.argsort(logypred_dens)[::-1][:1000]
logam_success_in_top_1000 = np.mean(np.array(ytest)[top_1000_logistic_logam])
print("Logistic GAM success in top 1000: ", logam_success_in_top_1000)

```

```

Logistic GAM misclassification rate: 0.11702412868632708
Logistic success in top 1000: 0.22
Logistic GAM success in top 1000: 0.345

```

```

In [10]: market_data = pd.read_csv('./marketing.csv')
        Y = np.where(market_data['y'] == 'yes', 1, 0)
        X = market_data.drop(['y'], axis = 1)
        X = pd.get_dummies(X)
        xtrain, xtest, ytrain, ytest = train_test_split(X, Y, test_size=0.33, random_state=1)
        #X.head()

```

```

In [12]: fit_RF = RandomForestClassifier(n_estimators = 500, oob_score = True, random_state=1)

```

```

In [13]: ypred_RF = fit_RF.predict(xtest)
        RD_MC_rate = np.mean(ypred_RF != ytest)
        print("Random forest misclassification rate: ", RD_MC_rate)
        oob_error = 1-fit_RF.oob_score_
        print("Random forest OOB error: ", oob_error)

```

```

Random forest misclassification rate: 0.12479892761394101
Random forest OOB error: 0.12980753359083552

```

The random forest misclassification rate is larger than base rate, logistic regression and logistic GAM classification rate. The random forest misclassification rate is smaller than OOB error.

(a)(ii)

```

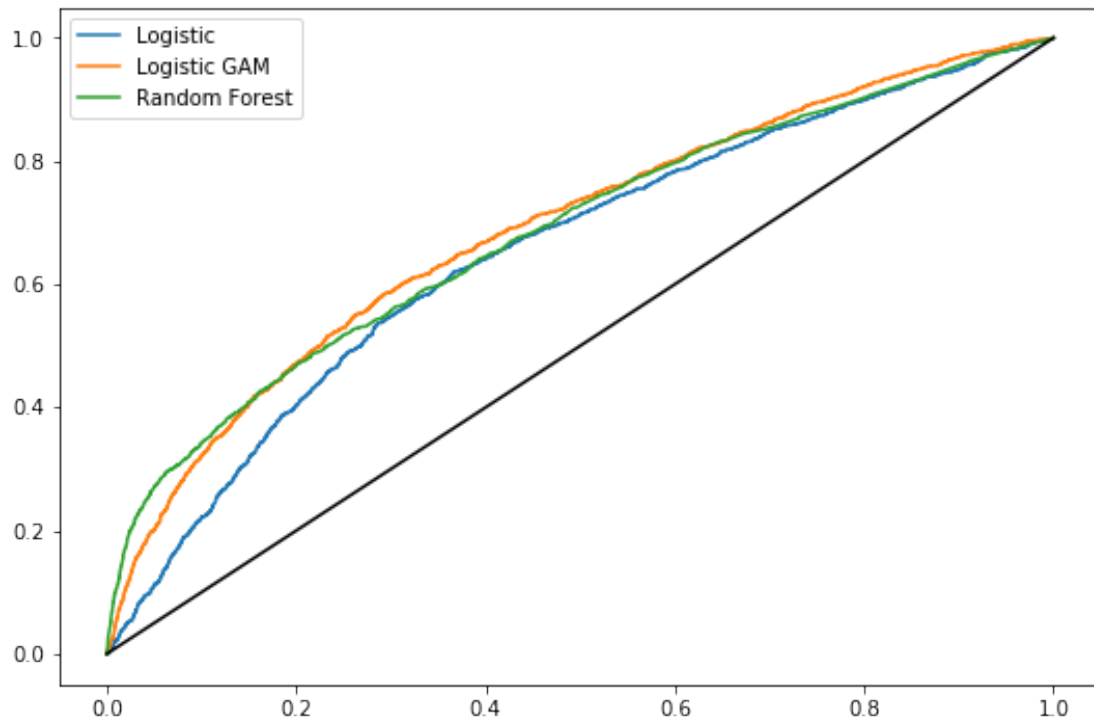
In [14]: test_prob_RF = fit_RF.predict_proba(xtest)[: ,1]
        roc_logit = roc_curve(ytest, test_prob)
        roc_logigam = roc_curve(ytest, logypred_dens)
        roc_rf = roc_curve(ytest, test_prob_RF)

        c_log, = plt.plot(roc_logit[0],roc_logit[1])
        c_logam, = plt.plot(roc_logigam[0],roc_logigam[1])
        c_RF, = plt.plot(roc_rf[0],roc_rf[1])
        line, = plt.plot([0,1],[0,1],color = 'black')

        plt.legend((c_log,c_logam,c_RF), ('Logistic','Logistic GAM','Random Forest'))
        plt.show()

```





```
In [15]: logistic_auc = roc_auc_score(ytest, test_prob)
logigam_auc = roc_auc_score(ytest, logypred_dens)
RF_auc = roc_auc_score(ytest, test_prob_RF)
print("AUC of logistic regression: ",logistic_auc)
print("AUC of random forest: ",RF_auc)
print("AUC of logistic GAM: ",logigam_auc)
```

```
AUC of logistic regression: 0.6520474585976039
AUC of random forest: 0.6829077915785764
AUC of logistic GAM: 0.68818487931642
```

The AUC of random forest is greater than that of logistic regression, but smaller than that of logistic gam.

(a)(iii)

```
In [16]: top_1000_RF = np.argsort(test_prob_RF)[:,-1][:1000]
RF_yes_in_top_1000 = np.mean(np.array(ytest)[top_1000_RF])
print("Random forest success in top 1000: ", RF_yes_in_top_1000)
```

```
Random forest success in top 1000: 0.433
```

The random forest top 1000 perform better than the other two classifiers, with success rate 0.433.

(b)

```
In [17]: # set the min samples leaf parameter to 10.
fit_RF_2 = RandomForestClassifier(n_estimators = 500, oob_score = True, random_state=

In [18]: ypred_RF_2 = fit_RF_2.predict(xtest)
RD_MC_rate_2 = np.mean(ypred_RF_2 != ytest)
print("New Random forest misclassification rate: ", RD_MC_rate_2)
oob_error_2 = 1-fit_RF_2.oob_score_
print("New Random forest OOB error: ", oob_error_2)
```

New Random forest misclassification rate: 0.11588471849865951

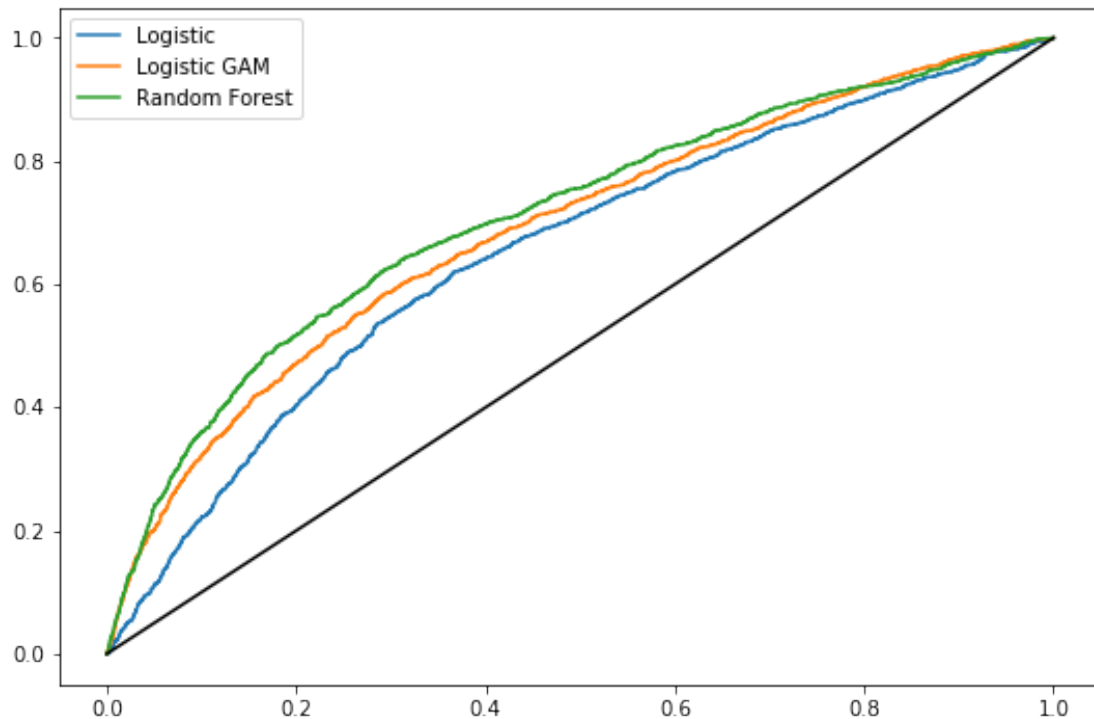
New Random forest OOB error: 0.117922815357697

The new random forest with minimum sample leaves = 10 has the smallest misclassification rate among logistic regression, logistic GAM and the old random forest. Also, the misclassification rate is smaller than the OOB error.

```
In [19]: test_prob_RF_2 = fit_RF_2.predict_proba(xtest)[: ,1]
roc_rf_2 = roc_curve(ytest, test_prob_RF_2)

c_log, = plt.plot(roc_logit[0],roc_logit[1])
c_logam, = plt.plot(roc_logigam[0],roc_logigam[1])
c_RF_2, = plt.plot(roc_rf_2[0],roc_rf_2[1])
line, = plt.plot([0,1],[0,1],color = 'black')

plt.legend((c_log,c_logam,c_RF), ('Logistic','Logistic GAM','Random Forest'))
plt.show()
logistic_auc = roc_auc_score(ytest, test_prob)
logigam_auc = roc_auc_score(ytest, logypred_dens)
RF_auc_2 = roc_auc_score(ytest, test_prob_RF_2)
print("AUC of logistic regression: ",logistic_auc)
print("AUC of logistic GAM: ",logigam_auc)
print("AUC of new random forest: ",RF_auc_2)
```



AUC of logistic regression: 0.6520474585976039  
AUC of logistic GAM: 0.68818487931642  
AUC of new random forest: 0.7090298625792812

The new random forest has the largest value of AUROC.

```
In [20]: top_1000_RF_2 = np.argsort(test_prob_RF_2)[:,-1][:1000]
         RF2_yes_in_top_1000 = np.mean(np.array(ytest)[top_1000_RF_2])
         print("Random forest success in top 1000: ", RF2_yes_in_top_1000)
```

Random forest success in top 1000: 0.382

The new random forest still gives better success rate in the top 1000 data set, which is better than the other two classifiers, but is not as good as the original tree.

(c)

```
In [21]: # class weight parameter to balanced
         fit_RF_3 = RandomForestClassifier(n_estimators = 500, oob_score = True,
                                         random_state=1,min_samples_leaf=10,class_weight="ba
```

```
In [22]: ypred_RF_3 = fit_RF_3.predict(xtest)
RD_MC_rate_3 = np.mean(ypred_RF_3 != ytest)
print("Balanced Random forest misclassification rate: ", RD_MC_rate_3)
oob_error_3 = 1-fit_RF_3.oob_score_
print("Balanced Random forest OOB error: ", oob_error_3)
```

```
Balanced Random forest misclassification rate:  0.22982573726541555
Balanced Random forest OOB error:  0.23544947344095601
```

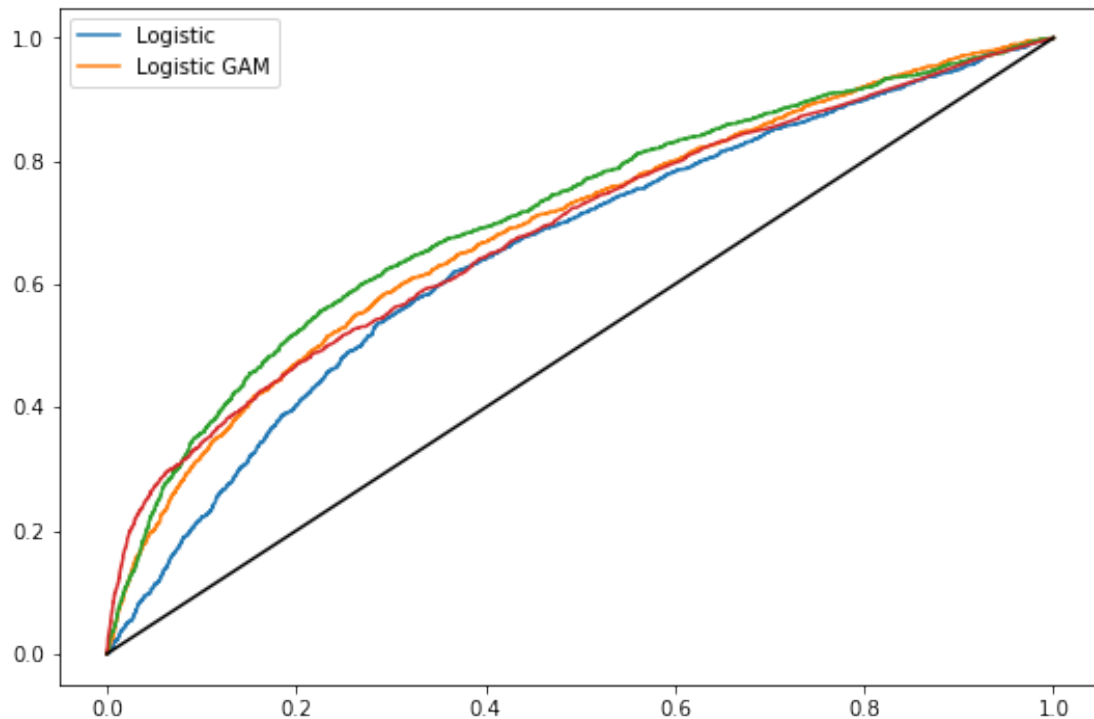
The new random forest with minimum sample leaves = 10 and class weight parameter set to balanced has larger misclassification rate than logistic regression, logistic GAM and the base rate. Also, the misclassification rate is smaller than the OOB error.

```
In [23]: test_prob_RF_3 = fit_RF_3.predict_proba(xtest)[:,-1]
roc_rf_3 = roc_curve(ytest, test_prob_RF_3)

c_log, = plt.plot(roc_logit[0],roc_logit[1])
c_logam, = plt.plot(roc_logigam[0],roc_logigam[1])
c_RF_3, = plt.plot(roc_rf_3[0],roc_rf_3[1])
c_RF = plt.plot(roc_rf[0],roc_rf[1])
line, = plt.plot([0,1],[0,1],color = 'black')

plt.legend((c_log,c_logam,c_RF), ('Logistic','Logistic GAM','Random Forest'))
plt.show()
logistic_auc = roc_auc_score(ytest, test_prob)
logigam_auc = roc_auc_score(ytest, logypred_dens)
RF_auc_3 = roc_auc_score(ytest, test_prob_RF_3)

print("AUC of logistic regression: ",logistic_auc)
print("AUC of logistic GAM: ",logigam_auc)
print("AUC of original random forest: ",RF_auc)
print("AUC of new random forest: ",RF_auc_2)
print("AUC of balanced random forest: ",RF_auc_3)
```



```
AUC of logistic regression: 0.6520474585976039
AUC of logistic GAM: 0.68818487931642
AUC of original random forest: 0.6829077915785764
AUC of new random forest: 0.7090298625792812
AUC of balanced random forest: 0.7099566816420015
```

The balanced random forest has the largest AUROC.

```
In [24]: top_1000_RF_3 = np.argsort(test_prob_RF_3)[:,-1][:1000]
         RF3_yes_in_top_1000 = np.mean(np.array(ytest)[top_1000_RF_3])
         print("Random forest success in top 1000: ", RF3_yes_in_top_1000)
```

```
Random forest success in top 1000: 0.385
```

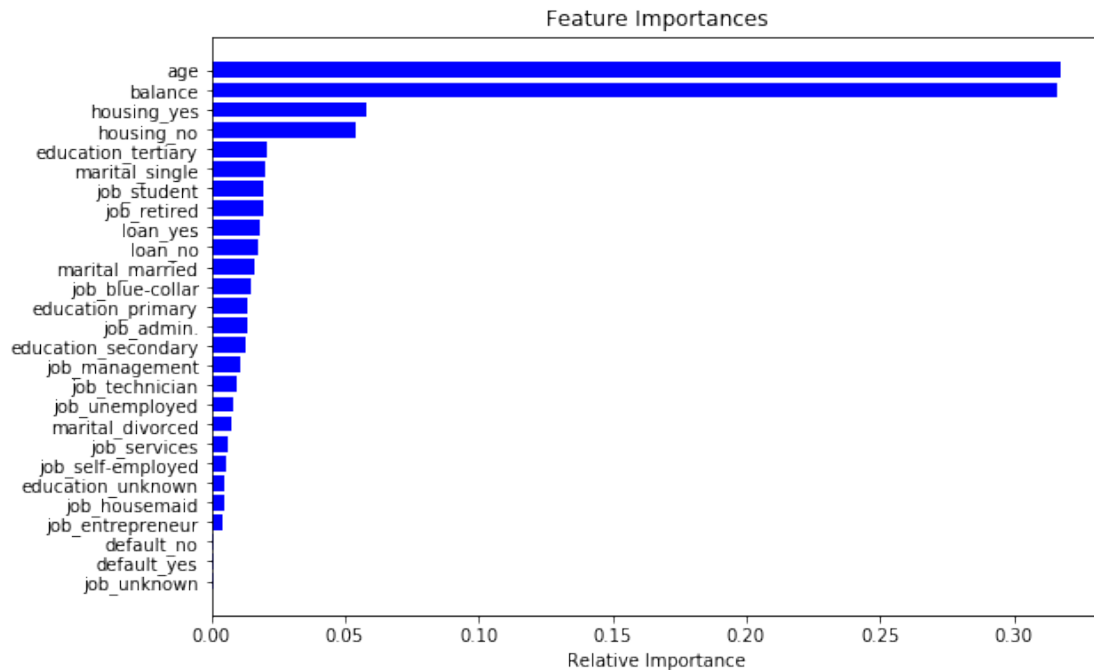
The balanced random forest has higher AUROC, but the misclassification rate is larger than the base rate. It is because we put more weights on the 'yes' class, so the classifier tends more to classify data into 'yes' class. In the case that in fact the 'yes' class are very rare, so the balanced random forest with higher weights on 'yes' class tends to make more mistakes.

### 3 Problem 3

(a) As we can see from the graph below, age and balance have the greatest relative importance.

```
In [25]: features = X.columns
importances = fit_RF_2.feature_importances_
idx = np.argsort(importances)

plt.title('Feature Importances')
plt.barh(range(len(idx)), importances[idx], color='b', align='center')
plt.yticks(range(len(idx)), [features[i] for i in idx])
plt.xlabel('Relative Importance')
plt.show()
```



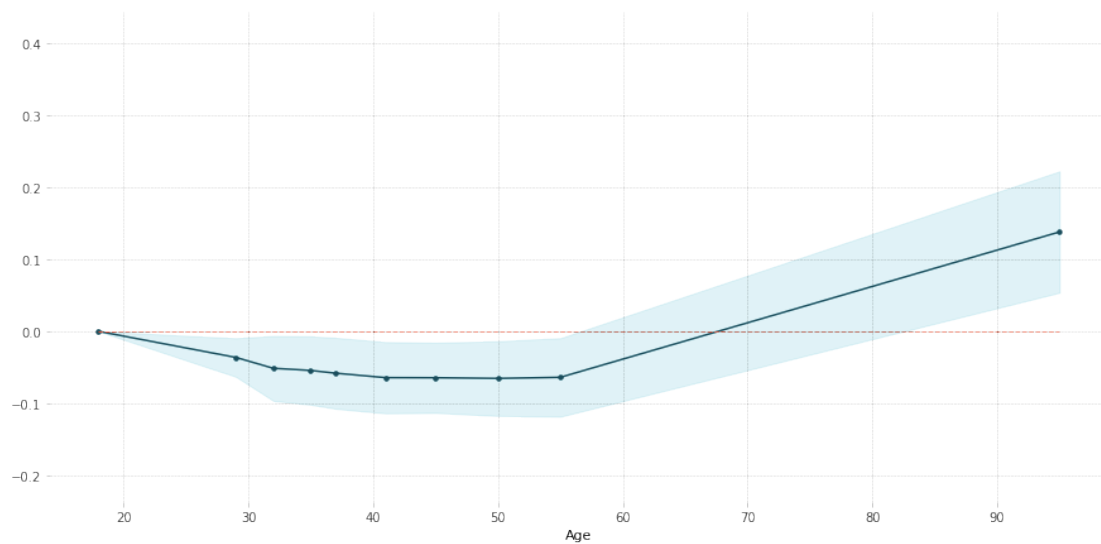
(b) Age: When exceeds 55, it has linear effects on the prediction. Balance: When exceeds 4000, the effect of balance stabilizes, before it reaches 4000, it has large effect on the prediction.

```
In [26]: # How do your predictions seem to depend on these variables, roughly?
#!pip install pdpbox
traindata = pd.concat([xtrain, pd.DataFrame(ytrain, index=xtrain.index)], axis=1)
from pdpbox import pdp
pdp_obj = pdp.pdp_isolate(model=fit_RF_2, dataset=traindata, model_features=features,
fig, axes = pdp.pdp_plot(pdp_obj, 'Age'))

pdp_obj_2 = pdp.pdp_isolate(model=fit_RF_2, dataset=traindata, model_features=features,
fig, axes = pdp.pdp_plot(pdp_obj_2, 'Balance'))
```

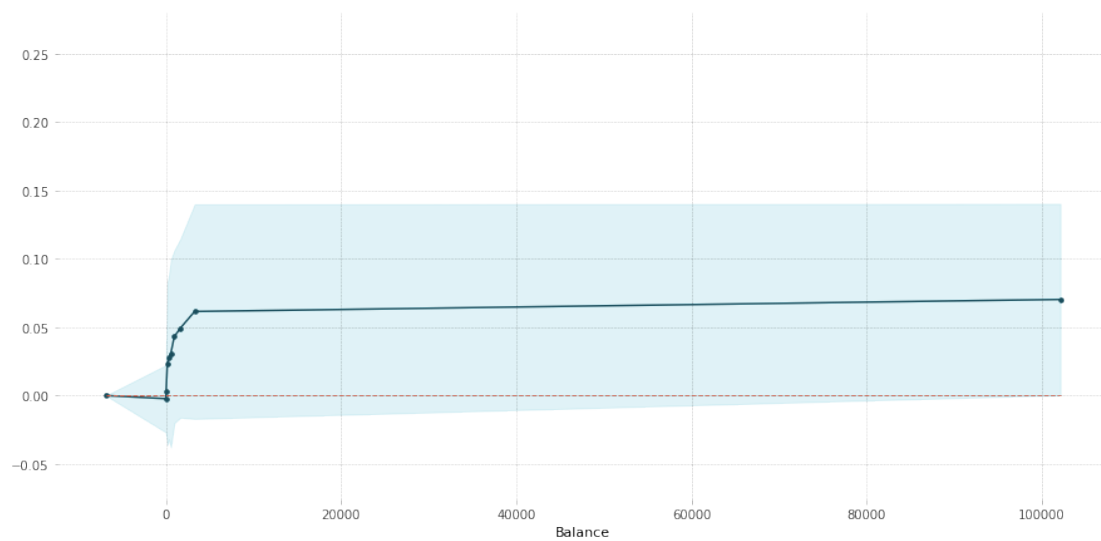
### PDP for feature "Age"

Number of unique grid points: 10



### PDP for feature "Balance"

Number of unique grid points: 10



## 4 Problem 4

(a)

```
In [27]: import scipy
```

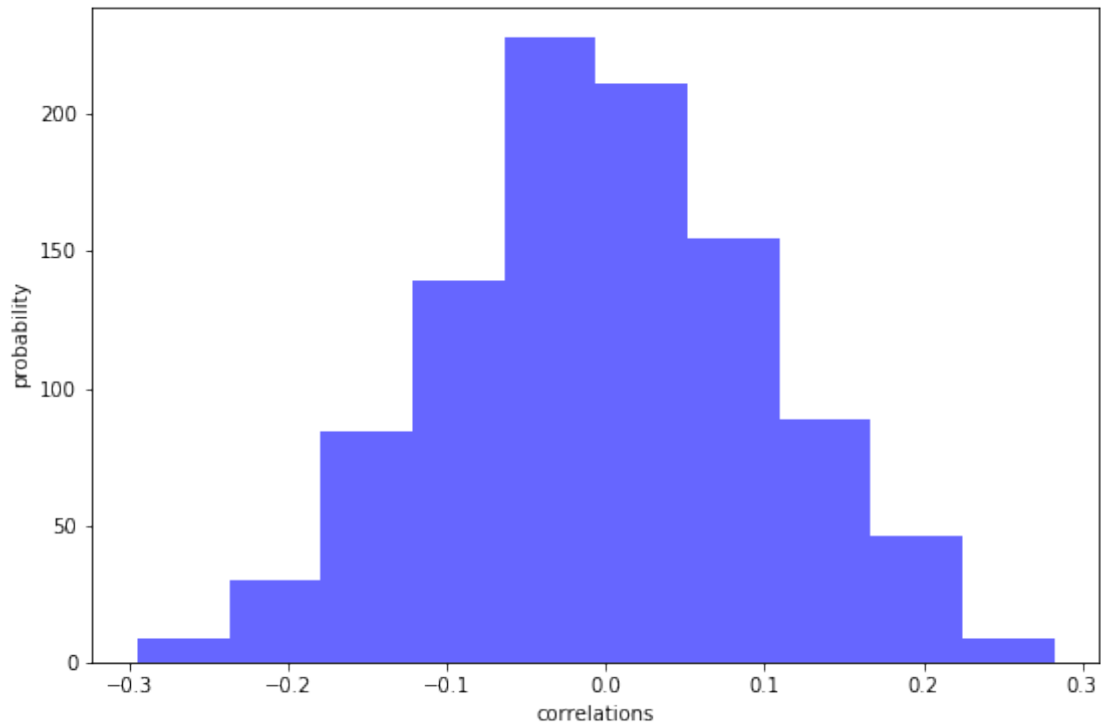
```
#You get to observe a bunch of data (that turns out to be worthless)
#Returns a data set with 100 variables and n data points
#The first 100 columns are the X variables (called X1,...X100)
#The last column is the response (called y)

def get_data(n):
    p=1000
    X = scipy.stats.norm().rvs(n*p).reshape((n,p))
    Y = scipy.stats.norm().rvs(n)
    return(X,Y)

#Generate some data
n = 100
X_train, y_train = get_data(n)
X_test, y_test = get_data(n)
train = get_data(n)
test = get_data(n)

res = np.apply_along_axis(scipy.stats.pearsonr, 0, X_train, y_train)
corr = res[0]
num_bins = 10
n, bins, patches = plt.hist(corr, num_bins, facecolor='blue', alpha=0.6)
plt.xlabel("correlations")
plt.ylabel('probability')
plt.show()
```





(b)

```
In [28]: idx = [abs(x)>0.2 for x in corr]
         X_good = X_train[:, idx]
```

(c)

```
In [29]: from sklearn.linear_model import LassoCV
         from sklearn.metrics import mean_squared_error
         #Split your X good data set into the first 0.8*n points (training set)
         # and the last 0.2*n points (test set)
         X_good_train = X_good[0:80,:]
         X_good_test = X_good[80:,:]

         fit_lasso = LassoCV().fit(X_good_train, y_train[0:80])

         model_mse = np.min(np.mean(fit_lasso.mse_path_, axis=1))
         print("Model MSE: ", model_mse)

         ypred_lasso = fit_lasso.predict(X_good_test)
         mse_las=mean_squared_error(ypred_lasso,y_train[80:])
         print("Test MSE: ", mse_las)
```

Model MSE: 0.31961243325165695

Test MSE: 0.30946531684335865

(d)

```
In [30]: X_good_test2 = X_test[:, idx]
         ypred_lasso2 = fit_lasso.predict(X_good_test2)
         mse_las2=mean_squared_error(ypred_lasso2,y_test)
         print("New data test MSE: ", mse_las2)
```

New data test MSE: 1.194013096054687

In [ ]: